Library Management Database SQL Statements

```
postgres=# CREATE DATABASE library;

CREATE DATABASE

postgres=# \c library

You are now connected to database "library" as user "postgres".

library=# CREATE TABLE authors (

library(# author_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

library(# author_name VARCHAR(120) NOT NULL,

library(# country VARCHAR(80)

library(# );

CREATE TABLE

library=# CREATE TABLE books (

library(# book_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

library(# title VARCHAR(200) NOT NULL,

library(# isbn VARCHAR(20) UNIQUE,

library(# category VARCHAR(80),

library(# publication_year INT,

library(# total_copies INT NOT NULL,

library(# available_copies INT NOT NULL,

library(# CHECK (total_copies >= 0),

library(# CHECK (available_copies >= 0),

library(# CHECK (available_copies <= total_copies)

library(# );

CREATE TABLE

library=# CREATE TABLE members (
```

```
library(# member_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
library(# member_name VARCHAR(120) NOT NULL,
library(# email VARCHAR(255) UNIQUE NOT NULL,
library(# phone VARCHAR(20) UNIQUE,
library(# joined_on DATE DEFAULT CURRENT_DATE
library(# );
CREATE TABLE
library=# CREATE TABLE book_authors (
library(# book_id INT REFERENCES books (book_id),
library(# author_id INT REFERENCES authors (author_id),
library(# PRIMARY KEY (book_id, author_id)
library(# );
CREATE TABLE
library=# CREATE TABLE borrowings (
library(# borrowing_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
library(# book_id INT REFERENCES books(book_id),
library(# member_id INT REFERENCES members(member_id),
library(# borrowed_on DATE DEFAULT CURRENT_DATE,
library(# due_date DATE NOT NULL,
library(# returned_on DATE,
library(# CHECK (due_date >= borrowed_on),
library(# CHECK (returned_on IS NULL OR returned_on >= borrowed_on)
library(# );
CREATE TABLE
library=# INSERT INTO authors (author_name, country)
```

```
library-# VALUES
library-# ('George Orwell', 'UK'),
library-# ('J.K. Rowling', 'UK'),
library-# ('Paulo Coelho', 'Brazil'),
library-# ('R.K. Narayan', 'India'),
library-# ('Agatha Christie', 'UK');
INSERT 0 5
library=# SELECT b.title, a.author_name
library-# FROM books b
library-# JOIN book_authors ba
library-# ON b.book_id = ba.book_id
library-# JOIN authors a
library-# ON ba.author_id = a.author_id
library-# WHERE a.author_name = 'George Orwell';
 title | author_name
-------+-------------
(0 rows)


library=# SELECT DISTINCT m.member_name, m.email
library-# FROM members m
library-# JOIN borrowings br
library-# ON m.member_id = br.member_id
library-# WHERE br.due_date < CURRENT_DATE
library-# AND br.returned_on IS NULL;
```

```
 member_name | email
-------------+-------
(0 rows)



library=# SELECT b.title,
library-# COUNT(br.borrowing_id) AS
library-# total_borrowed
library-# FROM books b
library-# JOIN borrowings br
library-# ON b.book_id = br.book_id
library-# GROUP BY b.book_id, b.title
library-# ORDER BY total_borrowed DESC
library-# LIMIT 3;
 title | total_borrowed
-------+----------------
(0 rows)



library=# SELECT m.member_id, m.member_name
library-# FROM members m
library-# WHERE NOT EXISTS (
library(# SELECT 1
library(# FROM borrowings br
library(# WHERE br.member_id = m.member_id
```

```
library(# );
 member_id | member_name
-----------+-------------
(0 rows)



library=# BEGIN;
BEGIN
library=*# WITH updated_book AS (
library(*# UPDATE books
library(*# SET available_copies = available_copies - 1
library(*# WHERE book_id = 2
library(*# AND available_copies > 0
library(*# RETURNING book_id
library(*# )
library-*# INSERT INTO borrowings
library-*# (book_id, member_id, borrowed_on, due_date)
library-*# SELECT book_id, 5, CURRENT_DATE,
library-*# CURRENT_DATE + 14
library-*# FROM updated_book;
INSERT 0 0
library=*# COMMIT;
COMMIT
library=# CREATE INDEX idx_borrowings_due_date
library-# ON borrowings(due_date);
```

CREATE INDEX

```
library=# EXPLAIN ANALYZE
library-# SELECT *
library-# FROM borrowings
library-# WHERE due_date < CURRENT_DATE;
                                                
```